

PDF Parser Pipeline

□ Visão Geral

O PDF Parser Pipeline é uma solução orientada a eventos desenvolvida para automatizar o processamento de documentos PDF em larga escala.

A aplicação recebe arquivos através de uma API REST construída com FastAPI, publica eventos em Apache Kafka e realiza o processamento assíncrono dos documentos através de workers especializados, permitindo escalabilidade horizontal e desacoplamento entre os componentes do sistema.

Este projeto foi desenvolvido com foco em arquiteturas modernas de Engenharia de Dados, Microsserviços e Processamento Distribuído.

□ Objetivo

Construir uma arquitetura escalável para processamento assíncrono de documentos PDF utilizando Apache Kafka e FastAPI, simulando cenários reais encontrados em ambientes corporativos de Engenharia de Dados e Back-end.

□ Problema Resolvido

Muitas organizações recebem diariamente centenas ou milhares de documentos em PDF contendo informações críticas para seus processos.

Entre os principais desafios estão:

- Processamento manual de documentos
- Gargalos em sistemas síncronos
- Baixa escalabilidade
- Dificuldade de integração com sistemas analíticos
- Necessidade de extração automática de dados

Este projeto demonstra uma arquitetura capaz de processar documentos de forma assíncrona, resiliente e escalável.



Arquitetura da Solução

Cliente -> FastAPI -> Apache Kafka -> PDF Worker -> Parser Engine -> JSON
Estruturado

Fluxo:

- Usuário envia um PDF para a API.
- A API armazena o arquivo.
- Um evento é publicado no Kafka.
- Um Worker consome a mensagem.

- O PDF é processado.
- Os dados extraídos são transformados em JSON.
- O resultado fica disponível para consumo por outros sistemas.

III Principais Benefícios

- Escalabilidade
 - Novos Workers podem ser adicionados sem alterar a API.
- Desacoplamento
 - O upload do documento é independente do processamento.
- Resiliência
 - Falhas no parser não impactam a disponibilidade da API.
- Performance
 - Permite processamento paralelo de múltiplos documentos.
- Observabilidade
 - Estrutura preparada para integração com logs, métricas e monitoramento.

□ Estrutura do Projeto

```
pdf-parser-pipeline/ | ├── app/ | ├── main.py | ├── kafka_producer.py | ├──
storage.py | ├── config.py | ├── worker/ | ├── consumer.py | ├──
pdf_parser.py | ├── data/ | ├── uploads/ | ├── results/ | ├── docker-compose.yml |
requirements.txt
```

□ Como Executar

Subir Kafka

```
docker compose up -d
```

Iniciar API

```
uvicorn app.main:app --reload
```

Iniciar Worker

```
python -m worker.consumer
```

Swagger

```
http://localhost:8000/docs
```

□ Casos de Uso

- Processamento de notas fiscais
- Processamento de contratos
- Extração de informações de currículos

- Digitalização de documentos
- Indexação para sistemas de IA
- Pipelines de ingestão documental

□ **Próximas Evoluções**

- OCR de documentos utilizando Tesseract para processamento de PDFs digitalizados.
- Armazenamento de documentos e resultados em Data Lake (S3, ADLS ou MinIO).
- Processamento distribuído com múltiplos consumidores Kafka para aumento de throughput.
- Implementação de Dead Letter Queue (DLQ) para tratamento de falhas.
- Estratégias de Retry automático para mensagens com erro.
- Extração inteligente de entidades como CPF, CNPJ, datas, valores monetários e informações contratuais.
- Classificação automática de documentos utilizando Machine Learning e LLMs.
- Integração com Apache Airflow para orquestração de pipelines.
- Observabilidade com Prometheus, Grafana e OpenTelemetry.
- Indexação dos documentos em banco vetorial para soluções RAG.
- Integração com modelos de IA para consultas em linguagem natural sobre documentos processados.
- Deploy em Kubernetes com escalabilidade horizontal automática.
- Implementação de autenticação e autorização utilizando JWT.
- Criação de API para consulta de status e histórico de processamento.
- Monitoramento de métricas operacionais e SLA dos consumidores Kafka.

⚙️ **Tecnologias Utilizadas**

- Backend
 - Python
 - FastAPI
 - Uvicorn
- Mensageria
 - Apache Kafka
 - Confluent Kafka Python
- Processamento de Documentos
 - PyMuPDF
- Infraestrutura
 - Docker

- Docker Compose
- Arquitetura
 - Event Driven Architecture (EDA)
 - Producer / Consumer Pattern
 - Asynchronous Processing
 - Microservices Ready

□ **Competências Demonstradas**

Este projeto demonstra conhecimentos em:

- Engenharia de Dados
- Arquiteturas Distribuídas
- Apache Kafka
- FastAPI
- Processamento Assíncrono
- Microsserviços
- APIs REST
- Docker
- Python
- Processamento de Documentos
- Arquiteturas Event Driven

Autor

Gabriel Sousa

Analista e Engenheiro de Dados com experiência em Business Intelligence, Engenharia de Dados, Automação de Processos e Arquiteturas Distribuídas.

Este projeto foi desenvolvido para explorar padrões amplamente utilizados em ambientes corporativos modernos, incluindo processamento assíncrono, mensageria e microsserviços.

🔗 Vamos nos conectar:

- LinkedIn: [Acessar](#)
- GitHub: [Acessar](#)

☆ Se o projeto foi útil, deixe uma estrela no repositório.